17. Модели на разпределени софтуерни архитектури. Среди и протоколи за разпределени приложения.
Изложението по въпроса трябва да включва следните по-съществени елементи:
1. Параметри на паралелната и разпределена обработка, метрика, методи за анализ.
2. Модели на разпределените софтуерни архитектури – процедурни, обектни, потокови, контекстни, йерархични, асинхронни и интерактивни модели на софтуерната архитектура. Структури, организация, компоненти, приложение.
3. Организация на разпределените приложения – клиент-сървер, двуслойни, трислойни и n-слойни модели. p2p. Сървери за приложения и Web-сървери. Метасистеми и грид. Сервизно-, моделно- и аспектно-ориентирани архитектури. Софтуерни агенти

1. **Параметри на паралелната и разпределена обработка, метрика, методи за анализ.**

Паралелна обработка:
Паралелната обработка наричаме обработка на данни, която се извършва от **няколко процеси или нишки**, които общо ще наричаме работници. Тя е **разпределена**, ако процесите са **разположени на няколко машини**, които общо ще наричаме изчислителни възли или само възли. При паралелната обработка целта е **ускорение**, а при разпределената – освен ускорение и **функционално разслояване**. Включва техники за разделяне на задачи между множество процесори или компютри за подобряване на ефективността и бързината на обработка.

Параметри на паралелната и разпределената обработка:

- **Паралелизмът** обикновено го отбелязваме с *p* и с него означаваме **броя на работниците**, които работят по време на изпълнение на дадена програма. Той се дели на два вида – **софтуерен и хардуерен**. Софтуерният паралелизъм е характеристика на програмите – **колко процеси/нишки се стартират**. Хардуерният паралелизъм зависи от **броя ядра/машини**, върху които се изпълнява задачата. Ускорението от софтуерния паралелизъм е ограничено от хардуерния – не може да очакваме 8 пъти ускорение от 8 нишки, ако машината има по-малко от 8 ядра, дори и ако едно ядро планира повече от една нишка.
- **Грануларност**: Грануларност е **степента на декомпозиция на данните**, т.е. на колко подзадачи се разбива цялата задача
    - **едра** - малък брой подзадачи. Малко време за разпределение на задачите на работниците, за сметка на лошо балансиране в средния случай
    - **фина** – голям брой подзадачи. Задачите са достатъчно голям брой, за да се счита, че всички работници ще получат по равен брой задачи в средния случай, но за сметка на много време по разпределението им;
    - **средна** - компромис между горните два вида. Това е златната среда, към която алгоритъмът за паралелна обработка трябва да се стреми.
- **Балансиране -** Балансирането е такова **разделение** на подзадачите, при което **всеки процес да получи равен на брой задачи** в очакване. Доброто балансиране ще гарантира, че в очакване подзадачите ще завършат по едно и също време. Това е желана цел, тъй като **времето за работа на един паралелен алгоритъм е равно на времето за работа на най-бавния работник в него**. Следователно не е желано един работник да приключи

работа много по-рано от друг. Балансирането може да се раздели по няколко оси:

- Време за осъществяване
  - статично
  - динамично
- Структура на разбиване
  - **централизирано** (топология звезда)
  - **разпределено** (напр. верига, решетка, хиперкуб)
  - **йерархично** (клъстери – дърво)

- **Синхронизация**: описва чрез какви механизми се осъществява комуникацията между процесите
  - **Асинхронни** - не обменят данни между различните си работници. Никой от процесите не знае за съществуването на останалите процеси
  - **Локално синхронни** - всеки работник си има дефинирани съседи, с които обменя информация. Приема информация от едни и предава на други, като тези от които приема и тези на които предава са точно дефинирани.
  - **Глобално синхронни** - всеки работник може да си комуникира с всеки останал (работниците образуват пълен граф)
- **Тип декомпозиция**: Налични са два вида декомпозиция на задачата:
  - **По данни** - SPMD (Single Program, Multiple Data)
  - **По управление** - MPMD (Multiple Program, Multiple Data)

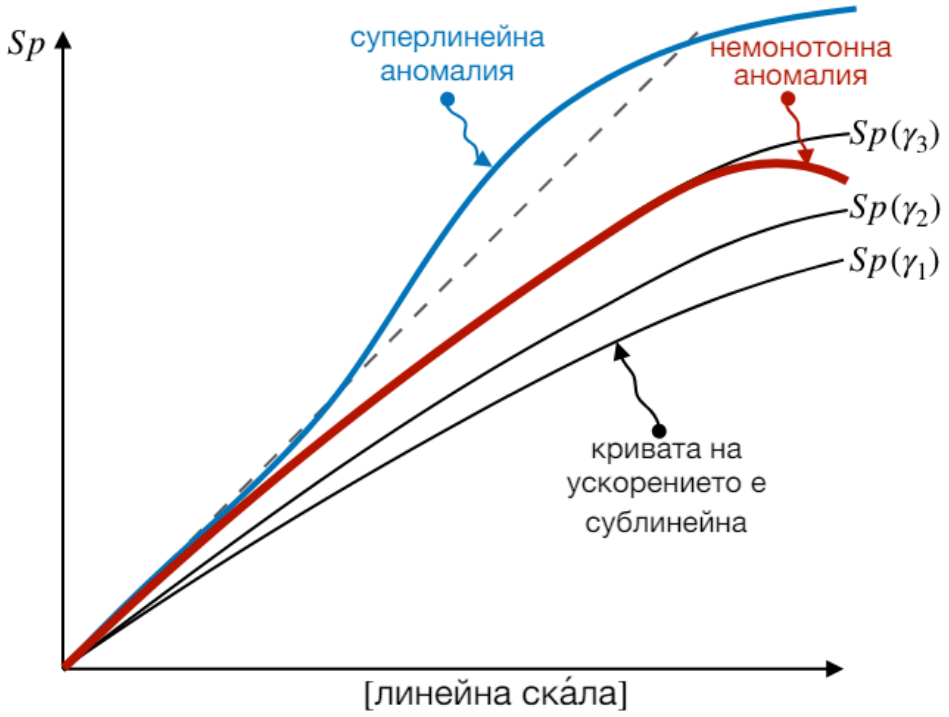
При SPMD използваме една и съща програма, която стартираме върху различни данни, а при MPMD имаме няколко различни програми.

Метрики:

- **Ускорение** - пресмятане чрез формулата: $Sp = T1 / Tp$, където $T1$ е времето за последователна обработка, Tp е времето за паралелна обработка при паралелизъм p. Поради наличие на комуникационни и синхронизационни забавяне, обикновено $1 < Sp < p$, но има и изключения
- **Ефикасност** - нормализирано ускорение, което пресмятаме чрез формулата: $Ep = Sp / p$, като обикновено стойностите са в интервала (0, 1) и затова е удобно да се измерва в проценти

Суперлинейна аномалия:
$Sp > p$. Например при голям размер на данните (надвишаващ капацитета на ОП) е възможно значително закъснение на последователната обработка поради бавни операции във външната памет
Нелинейна аномалия:
$S(p+1) < Sp$, програмата се забавя при увеличаване на паралелизма, което е контра-интуитивно

Графика на ускорението и ефикасността
Обикновено за дадена система се изобразяват фамилия от криви, които показват какво е нейното ускорение (и/или ефикасност), в зависимост от някакви параметри. Най-често използваният параметър е **грануларността**. Примери за графики, които включват и двете аномалии:



Сравнение между паралелна и разпределена обработка

Параметър / Тип обработка	Паралелна	Разпределена
Инфраструктура	мултипроцесор	мултикомпютър
Обмен на данни	Обща памет и механизъм за синхронизация	Предаване на съобщения чрез middleware
Обичайна декомпозиция	SPMD	MPMD
Примери за синхронни задачи	nBody, WaTor	Client-server, p2p, някои конвейери
Примери за асинхронни задачи	фрактали	някои конвейери

2. **Модели на разпределените софтуерни архитектури – процедурни, обектни, потокови, контекстни, йерархични, асинхронни и интерактивни модели на софтуерната архитектура. Структури, организация, компоненти, приложение.**

- **Процедурни модели** - Базирани на последователно изпълнение на процедури или функции. Често се използват в системи със строго определен ред на изпълнение на операциите. Също се базират на разпределена обработка чрез отдалечено извикване на процедури (remote procedure call (RPC)) или Remote Method Invocation (RMI). Представлява извикване на процедура на един възел от друг възел, като това е прозрачно за извикващия възел, тоест е като локална за него процедура
- **Обектни модели** - Обектно-ориентираните архитектурни модели се основават на разделянето на отговорностите на системата в индивидуални повторно

използваеми и независими обекти, които си сътрудничат. Основните принципи на тези модели са:

- **Абстракция** – опростяването на решението на реален проблем чрез дефиниране на подходящи класове;
- **Композиция** – изграждането на обекти от други обекти;
- **Наследственост** – автоматичното присвояване на функционалностите на едни обекти на други обекти и тяхното разширение и промяна;
- **Енкапсулация** – скриването на ненужните за потребителя детайли на някои обекти.
- **Полиморфизъм** – възможността на даден обект да се държи като друг, запазвайки логиката на собственото си поведение.

- **Потокови модели** - Потоковите архитектурни модели разглеждат системата като **последователност от трансформации** на постъпваща като от линеен поток информация. Всеки компонент **трансформира входните си данни в изходни**. Видове:
  - **Пакетна обработка (Batch sequential)** - Модулите се извикват в последователен ред един след друг, като изходът от един модул се предава като вход на следващия. **Отложена**, цел оптимално използване на ресурси
  - **Филтриране и канали (Pipe and Filter)** - позволява паралелизъм (конвейеризация). **Интерактивна**
  - **Контролни процеси (Process Control)** – като Pipe and Filter, но с добавени ограничения върху времето за изпълнение (real time constraints). Примери: управление на физически процеси, например при автомобилите и самолетите. **Реално време**
- **Контекстни (Data Centric) модели** - характеризират се с **централизирано хранилище за данните**, от където те са достъпни за всички останали компоненти на системата. Декомпозицията на модулите се състои от модул за управление на достъпа до данните и агенти, които извършват операции върху тях. Съществуват два основни контекстни модела:
  - **Хранилище (Repository) -** с активни (инициативни) агенти – хранилището е обикн. е организирано като СУБД, CORBA, UDDI или Web-услуги
  - **Черна дъска (Black board) -** с инициатива на модула данни – агентите са абонати за събития (event listeners), които настъпват при промяна в данните и на които абонатите отговарят реактивно
- **Йерархични модели** - Йерархичните архитектурни модели декомпозират системата на йерархични модули, като функциите се групират по йерархичен принцип на няколко нива. Координацията обикновено е между модули от различни нива (вертикална свързаност) и се базира на явни (т.е. “заявка-отговор”) обръщения. Ще разгледаме два основни типа йерархични модели
  - **Слоест модел** - При този модел, имаме последователност от слоеве, като всеки предоставя услуги на по-долните. Използван при много ОС като Unix, GNU и Windows. Също така протоколните стекове OSI и TCP/IP са слоести архитектури. Целта на този тип архитектура е бързо проектиране на (преносими) приложения.

    - **Master/Slaves** - Master/Slaves представлява разбиване на главна програма и подчинени програми. Главната програма разпределя работата на подчинените и след това е възможно да ги координира.
- **Асинхронни** - Асинхронните модели се базират на имплицитни асинхронни обръщения между обслужващите процеси, т.е. обмяна на съобщения – message passing
    - **Producer/Consumer** - Асинхронният обмен може да бъде от тип Producer/Consumer, където обменът на данни е 1:1. Има следните разновидности: интерактивен (online); отложен (offline)
    - **Publish/Subscriber** - Publish/Subscriber също е вид асинхронен обмен на съобщения, където обменът на данни е 1:много. Има следните разновидности: *topic-based; context-based*
- **Интерактивни модели** - Интерактивните модели поддържат **интензивен потребителски интерфейс**. За целта декомпозицията на системата е на 3 функционални модула:
    - **модул за представяне (изглед)**, имплементиращ потребителския интерфейс. Използва се за представяне (в това число графично и мултимедийно) на изходните данни и намеса на потребителите в обработката (т.е. вход за данни и контрол);
    - **модул на данни**, поддържащ данновия модел заедно с базова функционалност за обработката на данните;
    - **модул за управление**, който отговаря за системни комуникации, управление на процесите, инициализиране и конфигуриране на модули данни, както и управление на изгледи.

    две категории: PAC (Presentation-Abstraction-Control) и MVC (Model-View Controller)

3. **Организация на разпределените приложения – клиент-сървер, двуслойни, трислойни и n-слойни модели. p2p. Сървери за приложения и Web-сървери. Метасистеми и грид. Сервизно-, моделно- и аспектно-ориентирани архитектури. Софтуерни агенти**

- **Клиент сървър** - Клиент-сървър е класически MPMD модел за адаптиране на ограничена клиентска инфраструктура към сложни задачи. Традиционен модел, където клиентите изпращат заявки към сървъра, който обработва заявките и връща резултати. Процесите се класифицират като два основни:
    - **Сървър**, който е реактивен процес, имплементиращ приложната логика и данновия модел под формата на конкретен набор от услуги.
    - **Клиент**, който е активен интерфейсен процес, използващ услугите на сървъра.

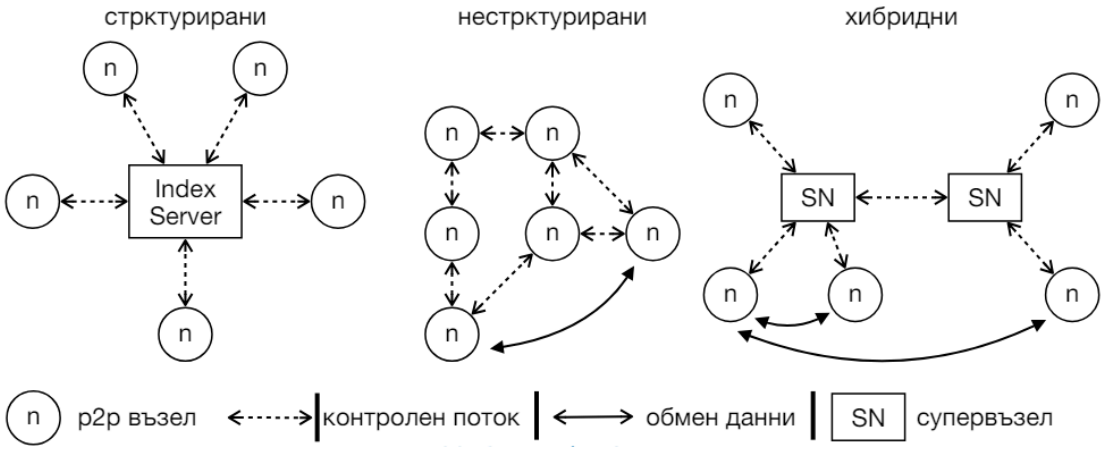
Сървърите са специално проектирани устройства, направени да обработват определен товар. Има два основни вида клиенти – т**ънки (thin)**, които делегират цялата работа на сървъра, и **дебели (fat)**, които изпълняват част от задачата. Клиент-сървър модела е много разпространен, тъй като той е заложен в основата на IP протокола.

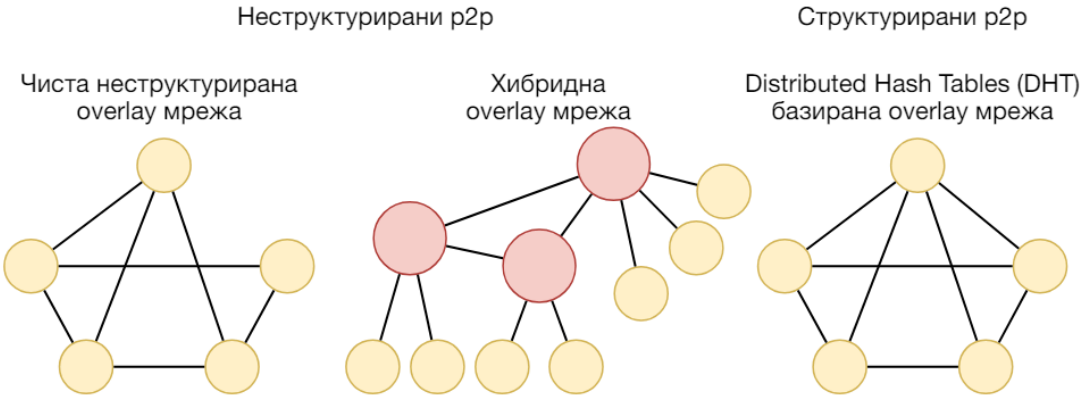
- **Двуслойни модели** -Двуслойната архитектура включва клиентски слой и сървърен слой. Логиката на приложението е разделена между тези два слоя, като клиентът се грижи за потребителския интерфейс, а сървърът обработва бизнес логиката и данните. Пример за двуслойни модели са традиционните

файл-сървър системи, където файловете се съхраняват и управляват централизирано на сървъра, а клиентите имат достъп до тях.

- **Трислойна архитектура** - При трислойната архитектура сървърната част се декомпозира допълнително на сървър на приложението, в което се обработват заявките, и сървър на данните.
    - **Клиентски слой (Presentation Layer)**: Отговаря за потребителския интерфейс.
    - **Слой на приложението (Application Layer)**: Съдържа бизнес логиката на приложението.
    - **Слой на данните (Data Layer)**: Управлява достъпа и съхранението на данни.
- **N-слойна архитектура** - компонентите са разделени на няколко слоя. Можем да си представим, че слоевете са вертикално подредени и **компонентите на даден слой са клиенти на сървъри от съседния долен слой**
  При N-слойните архитектури редовно се срещат многослойни сървъри, където сървърната част е декомпозирана на поне два слоя, които се класифицират като: междинни слоеве, които се намират между интерфейсния и вътрешния слой, имплементиращ бизнес логиката; вътрешен слой, най-долния слой, който обичайно се грижи за данните или се използва за комуникации
- **Peer-to-peer архитектури** - Модел, при който всички участници са равноправни и споделят ресурси директно помежду си. Пример: BitTorrent. Всеки участник е еднакво привилегирован и заделя част от ресурсите си (cpu, hd и други), която да се използва от останалите участници в мрежата без нуждата от каквато и да е централна координация. За разлика от класическата архитектура клиент-сървър, при p2p **всеки участник изпълнява функцията и на клиент на сървър за другите участници**. Характеризира се с: **децентрализирана топология; самоорганизация; скалируемост; отказоустойчивост; анонимност**
    - **Неструктурирани p2p** - При тях **топологията на свързване е произволна**, както и производителността на комуникационните канали. Често се случва възли да се включват и отпадат. В резултат, **общата производителност е също произволна**
    - **Структурирани p2p** - Топологията е изрично зададена, което дава гаранции и за **бързодействието**. Съществуват няколко топологии, като много популярна е топологията **хорда**. **При нея всеки възел се свързва с фиксиран брой съседи**
    - **Хибридни/йерархични** - Ниските нива са структурирани, като всяко ниско ниво играе като супервъзел в неструктурирана мрежа

3.2.4. Диаграми





- **Сървъри за приложения**: Използват се за обработка на бизнес логика и управление на приложни услуги. Пример: Apache Tomcat, JBoss.
- **Web-сървъри**: Отговарят за предоставянето на уеб съдържание на клиентите чрез протоколи като HTTP и HTTPS. Пример: Apache HTTP Server, Nginx.
- **Метасистеми**: Системи, които обединяват ресурси от различни компютърни мрежи, за да осигурят общо изчислително пространство. Пример: Globus Toolkit.
- **Грид**: Специализиран вид метасистема, където множество компютри работят съвместно за решаване на сложни задачи, като същевременно споделят ресурси.
- **Сервизно-ориентирана архитектура (SOA)**: Използва услуги като основни градивни блокове, които комуникират чрез добре дефинирани интерфейси и протоколи. Пример: Web Services (SOAP/REST).
- **Микроуслуги**: Архитектурен стил, който структурира приложение като набор от малки, автономни услуги, всяка от които изпълнява специфична бизнес функция. Пример: Docker и Kubernetes за управление на микроуслуги.
- **Софтуерни агенти -** извършват задачи в разпределена среда. Те могат да комуникират помежду си, да вземат решения и да се адаптират към променящи се условия